

EC2 instance types analysis

Comparison matrix

First principles: Cost per hour vs performance
vs availability needs

Workload use cases

Web Servers / Microservices

- M (General Purpose) family
- Balanced: 1 vCPU : 4 GB RAM
- Savings Plan / Reserved: If traffic is steady
- Auto-Scaling: Mix of On-Demand & Spot for spikes

Databases (SQL / NoSQL)

- R (Memory Optimized) family
- High RAM: 1 vCPU : 8 GB RAM
- Reserved Instances: Critical for stable, "always-on" data layers. Avoid spot.

Batch Processing / Analytics

- C (Compute Optimized)
- High CPU: 1 vCPU : 2 GB RAM
- Spot instances: Best fit. Fault-tolerant jobs save ~70-90% here

Dev / Test Environments

- T (Burstable)
- Variable: Baseline % + Burst
- Spot or On-Demand: Turn off when not in use (even cheaper)

AI / ML Training

- P / G (Accelerated) family
- GPU: Dedicated Graphics / Tensor
- Spot (if reasonable): GPU instances are expensive; Spot cuts cost massively

Real Scenario:

You're running a web application. Traffic is unpredictable. Which instance + pricing model?

I'm running a general instance (probably M family) on a reserved instance pricing model with burst instances for when traffic spikes up. (Temp horizontal scaling)

Costs are brought down when I use reserved instances and when traffic picks up I can temporarily scale my resources horizontally with spot instances on an as-needed basis coordinated through an Auto Scaling Group (this addresses availability as well).

Performance wise a web application in general is not a CPU / memory / GPU intensive workload so a general purpose instance as a jack of all trades does the job here.

Vertical Scaling Trade-offs

Vertical scaling (bigger instances):

- Is simple
- Has hard limits
- Increases blast radius
- Scales slowly and expensively

It works for predictable, statful workloads
— but fails under unpredictable demand

Statful vs Stateless workloads

Statful

A statful system stores session or application state locally on the instance

Characteristics

- Data tied to a specific machine
- Harder to scale horizontally
- Failure = possible data / session loss

Examples

- Databases
- In-memory session storage
- Legacy apps with local user sessions

Implications

- Needs sticky sessions
- Avoid spot instances
- Favours vertical scaling or careful replication

Stateless

A stateless system keeps no client or session state on the instance

Characteristics

- Any request can go to any instance
- Easy horizontal scaling
- Instances are disposable

Examples

- REST APIs
- Web servers using external session stores
- Microservices

Implications

- Works well with Auto Scaling
- Safe to use Spot instances
- Enable high availability and elasticity

Why Elasticity Matters More Than Power

Elasticity lets systems:

- Absorb traffic spikes
- Recover from failures automatically
- Match resources to demand in real time

Many small, replaceable instances outperform one powerful server when traffic is unpredictable

How Pricing Models Shape Architecture

Pricing choices force design decisions:

- On-Demand → encourages elasticity and right-sizing
- Reserved → favours stable, always-on workloads
- Spot → requires stateless, fault-tolerant design